$F2 \parallel WC_{\max}$ Structural Properties and Dynamic Programming

Author One[a,*], Author Two[b]

[a]Institution One, Location, Country

[b]Institution Two, Location, Country

Abstract

Minimizing $WC_{\max}$ on a two-machine flow shop, denoted $F2 \parallel WC_{\max}$, is a classical NP-hard scheduling problem. Existing exact methods rely on enumerative techniques that scale poorly with instance size, while approximation algorithms often impose restrictive assumptions on processing times. We first sequence all jobs globally by Johnson's rule, obtaining an initial schedule π. Scanning π from left to right, we partition it into at most K blocks: whenever a weight not seen before appears, it serves as the final job of the current block, and the next job starts a new block. We establish structural properties of optimal schedules: the permutation property, the positional constraint that block $\mathcal{G}_k$ jobs cannot appear beyond the first k blocks, and the block-end condition requiring the final job of each block to carry that block's weight (automatically satisfied by construction). The intra-block Johnson ordering inherited from π is further formalized within the DP framework. Building on these properties and the interval-based dynamic programming framework (Hall, 1994) developed for $F2|r_j|C_{\max}$, we design a polynomial-time dynamic programming algorithm with $O(|Y|_{\max} \cdot n)$ complexity that appends jobs block by block. We further show that geometric rounding of weights and processing times converts this algorithm into a polynomial-time approximation scheme.

Keywords: Flow shop scheduling; $WC_{\max}$; Dynamic programming; Structural properties

1 Introduction

2 Preliminaries

We consider the problem $F2 \parallel WC_{\max}$. Let $\mathcal{J} = \{J_1, J_2, \ldots, J_n\}$ be the set of n jobs to be processed on two machines M_1 and M_2 in series: each job J_j must be processed first on

*Corresponding author. E-mail address: author@email.com

M_1 for a_j time units and then on M_2 for b_j time units, with no preemption allowed. M_1 and M_2 can process at most one job at a time. Each job J_j has a weight $w_j > 0$. A schedule $\pi = (\pi(1), \pi(2), \ldots, \pi(n))$ specifies the processing order on both machines. For a schedule π, let $A_j = \sum_{h=1}^{j} a_{\pi(h)}$ and $B_j = \sum_{h=1}^{j} b_{\pi(h)}$ be the cumulative M_1 and M_2 completion times after the first j jobs.

The objective is to find a schedule π that minimizes $WC_{\max}$.

For $F2 \parallel WC_{\max}$, a permutation schedule in which the job order on M_1 and M_2, Possess an identical processing order for jobs (Baker, 1974). Johnson's rule (Johnson, 1954) sequences all n jobs to minimize the makespan: jobs with $a_j \leq b_j$ go first in non-decreasing order of a_j, and the remaining jobs follow in non-increasing order of b_j.

Lemma 2.1. *For $F2 \parallel WC_{\max}$, there exists an optimal schedule in which all jobs are sequenced according to Johnson's rule.*

Proof. Consider any optimal schedule π^* for $F2 \parallel WC_{\max}$. If the jobs in π^* are already in Johnson order, we are done. Otherwise, there exist two adjacent jobs J_u and J_v (with J_u immediately preceding J_v) that violate Johnson's rule, i.e., $\min\{a_u, b_v\} > \min\{a_v, b_u\}$.

Let π' be the schedule obtained by swapping J_u and J_v. For any permutation schedule σ, its makespan can be expressed as

$$C(\sigma) = \max_{1 \leq r \leq n} \Big\{ \sum_{h=1}^{r} a_{\sigma(h)} + \sum_{h=r}^{n} b_{\sigma(h)} \Big\}.$$

A standard exchange argument shows that when $\min\{a_u, b_v\} > \min\{a_v, b_u\}$, swapping J_u and J_v does not increase the makespan, i.e., $C(\pi') \leq C(\pi^*)$. Moreover, since the two jobs are adjacent on both machines and the swap only affects these two positions, all other jobs finish no later in π' than in π^*. Hence for every job J_j, its completion time $C_j(\pi') \leq C_j(\pi^*)$, which implies $w_j C_j(\pi') \leq w_j C_j(\pi^*)$ for all j. Therefore $WC_{\max}(\pi') \leq WC_{\max}(\pi^*)$. By repeatedly applying this exchange argument to adjacent inversions, any schedule can be transformed into a Johnson-ordered schedule without increasing the objective value. Thus, there exists an optimal schedule in which all jobs are sequenced according to Johnson's rule. □

2.1 Mixed Integer Linear Programming Formulation

The problem $F2 \parallel WC_{\max}$ can be formulated as a mixed integer linear program (MILP). For a permutation schedule, let the positions be indexed by $k = 1, \ldots, n$. We introduce the following decision variables:

- $x_{jk} \in \{0, 1\}$ — equals 1 if job J_j occupies position k, and 0 otherwise;
- $C_k^1 \geq 0$ — completion time of the job at position k on M_1;

- $C_k^2 \geq 0$ — completion time of the job at position k on M_2;
- $C_j \geq 0$ — completion time of job J_j on M_2;
- $Z \geq 0$ — the maximum weighted completion time $WC_{\max}$.

Let $M = \sum_{j=1}^{n}(a_j + b_j)$ be a sufficiently large constant. The MILP model is:

$$\min \quad Z \tag{1}$$

$$\text{s.t.} \quad \sum_{k=1}^{n} x_{jk} = 1, \quad \forall\, j \in \mathcal{J} \tag{2}$$

$$\sum_{j=1}^{n} x_{jk} = 1, \quad \forall\, k = 1, \ldots, n \tag{3}$$

$$C_1^1 = \sum_{j=1}^{n} a_j x_{j1} \tag{4}$$

$$C_k^1 = C_{k-1}^1 + \sum_{j=1}^{n} a_j x_{jk}, \quad \forall\, k = 2, \ldots, n \tag{5}$$

$$C_1^2 = C_1^1 + \sum_{j=1}^{n} b_j x_{j1} \tag{6}$$

$$C_k^2 \geq C_k^1 + \sum_{j=1}^{n} b_j x_{jk}, \quad \forall\, k = 2, \ldots, n \tag{7}$$

$$C_k^2 \geq C_{k-1}^2 + \sum_{j=1}^{n} b_j x_{jk}, \quad \forall\, k = 2, \ldots, n \tag{8}$$

$$C_j \geq C_k^2 - M(1 - x_{jk}), \quad \forall\, j \in \mathcal{J},\ k = 1, \ldots, n \tag{9}$$

$$Z \geq w_j C_j, \quad \forall\, j \in \mathcal{J} \tag{10}$$

$$x_{jk} \in \{0, 1\}, \quad C_k^1, C_k^2, C_j, Z \geq 0 \tag{11}$$

Constraint (2)–(3) ensure a one-to-one assignment between jobs and positions. Constraints (4)–(5) compute the completion time of each position on M_1. Constraints (6)–(8) compute the completion time of each position on M_2: for $k \geq 2$, the job at position k starts on M_2 only after it completes M_1 and the preceding job releases M_2. Constraint (9) links the position-based completion times to the job-based completion times via a big-M formulation: when $x_{jk} = 1$, job J_j is at position k and $C_j \geq C_k^2$; when $x_{jk} = 0$, the right-hand side becomes $C_k^2 - M \leq 0$, which is non-binding. Constraint (10) enforces $Z \geq w_j C_j$ for every job, so that minimizing Z yields the optimal $WC_{\max}$.

3 Problem formulation

Theorem 3.1. *$F2 \parallel WC_{\max}$ is NP-hard.*

Proof. We reduce from the Partition problem: given n positive integers $x_1, x_2, \ldots, x_r$ with $\sum_{i=1}^{r} x_i = 2T$, does there exist subset Z_1 and $Z_2 \subseteq \{1, \ldots, r\}$ such that: $\sum_{i \in Z_1} x_i = \sum_{i \in Z_2} x_i = T$?

Construct an instance of $F2 \parallel WC_{\max}$, Let the number of jobs $n = r + 1$. Let $a_j = 1$, $b_j = rx_j$, $w_j = T + 1$ for jobs $j = 1, \ldots, n-1$. Put $a_n = rT$, $b_n = 1$, $w_n = 2T + 1$ for job J_n. Does there exist a threshold Y such that:

$$WC_{\max} \leq Y = r(T+1)(2T+1)$$

holds?

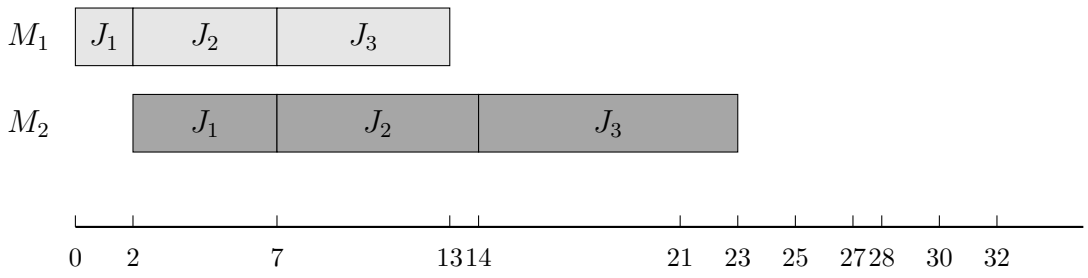

Figure 3.1

We partition jobs j_1 to j_{n-1} into two subsets via j_n, the distribution of jobs on the machine is$(\mathcal{J}_{j \epsilon A_1}, J_n, \mathcal{J}_{j \epsilon A_2})$. Since the processing times of A_1 and A_2 are unknown, while their total processing time is 2T. We may assume that the processing time of A_1 on M_2 is $r(T + \xi)$, A_2 is $r(T - \xi)$, where $-T \leq \xi \leq T$. Therefore, the completion time of J_n is determined by the following formula:

$$C_n^1 = |A_1| + rT + 1 \tag{12}$$

$$C_n^2 = 1 + r(T + \xi) + 1 \tag{13}$$

and $C_n = \max\{C_n^1, C_n^2\}$

If $\xi = 0$, then $\sum_{j \epsilon A_1} b_j = \sum_{j \epsilon A_2} b_j$, which means the partition problem has a feasible solution. In this case, the completion time of job J_n is determined by (12), we have:

$$\begin{aligned} w_n C_n &= (2T+1)(|A_1| + rT + 1) \\ &\leqslant (2T+1)(rT + r) \\ &= r(T+1)(2T+1) \\ &= Y \\ WC_{\max} &= r(T+1)(C_n + rT) = Y \end{aligned}$$

so $w_j C_j \leqslant WC_{\max} = Y$

If the scheduling problem exists a threshold y such that $WC_{\max} \leq y$. When $\xi > 0$, $C_n$is determined by (13), we have:

$$\begin{aligned} w_n C_n &= (1 + r(T + \xi) + 1)(2T + 1) \\ &\geqslant (1 + r(T + 1) + 1)(2T + 1) \\ &> Y \end{aligned}$$

When $\xi < 0$, $C_n$is determined by (12), we have:

$$\begin{aligned} WC_{\max} &= (C_n + \sum_{j \epsilon A_2} b_j)(T + 1) \\ &= (1 + rT + |A_1| + r(T - \xi))(T + 1) \\ &\geqslant (rT + r + rT)(T + 1) \\ &= r(T + 1)(2T + 1) = Y \end{aligned}$$

so $WC_{\max} > Y$, there exists a contradiction.

Therefore, the Partition instance admits a solution if and only if the constructed $F2 \parallel WC_{\max}$ instance admits a schedule with $WC_{\max} \leq Y$.

□

4 $F2 \parallel WC_{\max}$ for the number of weights is constant

When the number of distinct weights K is bounded by a constant, the structural properties established in Section 2 enable a polynomial-time dynamic programming algorithm. By Lemma 2.1, the initial job sequence follows Johnson's rule; the block partition and all subsequent steps are defined with respect to this Johnson order. We first describe the block partition and group logic underlying our approach, then present the DP formulation, and finally give the complete algorithm and its complexity analysis.

4.1 Algorithm Properties and Dynamic Programming

Starting from the first job, a new block is created whenever a weight value not encountered before appears, that job becomes the final job of the current block, and the next job starts a new block. Jobs of the same weight form a natural group, denoted $\mathcal{G}_1, \mathcal{G}_2, \ldots, \mathcal{G}_K$. Since each block introduces at least one previously unseen weight, the procedure yields at most K blocks, denoted $\mathcal{B}_1, \mathcal{B}_2, \ldots, \mathcal{B}_K$. By construction, the final job of each block carries the weight that triggered the cut. By construction, block weights are non-increasing, i.e., $W(\mathcal{B}_1) \geq W(\mathcal{B}_2) \geq \cdots \geq W(\mathcal{B}_K)$.

Example 4.1. *Consider a 6-job instance with Johnson sequence $\pi = (J_1, \ldots, J_6)$ and jobs with (aj, bj, wj) values:* $(J_1, \ldots, J_6) = (2, 5, 2); (5, 7, 2); (6, 9, 5); (8, 4, 2); (4, 3, 4); (3, 2, 2);$.

Scanning π from left to right and cutting whenever a weight not seen before appears yields the following partition: The resulting blocks are $\mathcal{B}_1 = \{J_1, J_2, J_3\}$, $\mathcal{B}_2 = \{J_4, J_5\}$, $\mathcal{B}_3 = \{J_6\}$. And $\mathcal{G}_1 = \{J_3\}$, $\mathcal{G}_2 = \{J_5\}$, $\mathcal{G}_3 = \{J_1, J_2, J_4, J_6\}$.

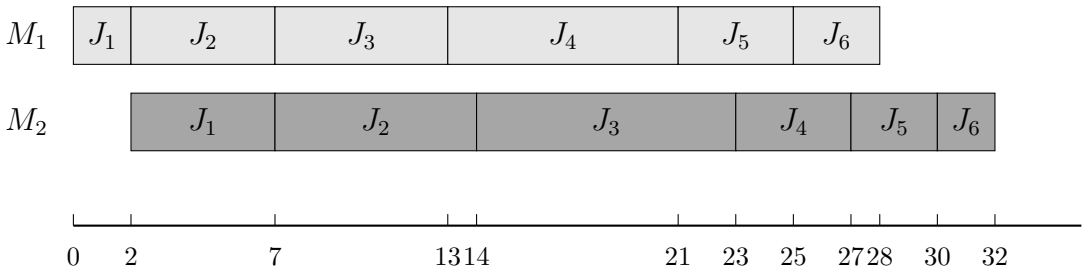

Figure 4.1

Lemma 4.1. *For a given schedule π, rescheduling each blocks for $\mathcal{B}_1, \mathcal{B}_2, \ldots, \mathcal{B}_K$ to Johnson's algorithm and concatenating blocks schedules delivers a schedule of length at most $WC(\pi)$*

Proof. □

We now develop a dynamic programming algorithm that processes jobs sequentially in Johnson order and assigns each job to one of the K blocks $\mathcal{B}_1, \ldots, \mathcal{B}_K$.

Lemma 4.1 provides the structural basis: it guarantees that, given any assignment of jobs to the K blocks, sequencing the jobs within each block $\mathcal{B}_i$ according to Johnson's rule and concatenating the blocks in non-increasing weight order yields a schedule whose makespan is no larger than that of any other schedule respecting the same assignment. We denote by $\mathscr{H}$ the set of states maintained during the computation; each state summarizes the aggregate parameters of all K blocks after a prefix of jobs has been processed.

State Definition. A state is a $4K$-tuple

$$(C_1, \ldots, C_K;\ A_1, \ldots, A_K;\ B_1, \ldots, B_K;\ L_1, \ldots, L_K) \in \mathscr{H}$$

for $\mathcal{B}_i$, $i = 1, \ldots, K$:

- A_i — total processing time on machine M_1 of jobs in block $\mathcal{B}_i$;
- B_i — total processing time on machine M_2 of jobs in block $\mathcal{B}_i$;
- C_i — makespan of block $\mathcal{B}_i$ when the jobs assigned to it are scheduled in isolation;
- L_i — weight of the last job in block $\mathcal{B}_i$.

The initial state set contains a single state with all components set to zero: $\mathscr{H}^{(0)} = \{(0, \ldots, 0)\}$.

State Transition. For $j = 1.....n$ perform the following computations. Set $Y_i \leftarrow \emptyset, \quad i = 1, \ldots, k$. For each tuple $(C_1, \ldots, C_K;\ A_1, \ldots, A_K;\ B_1, \ldots, B_K;\ L_1, \ldots, L_K) \in \mathscr{H}^{(j-1)}$ and $i = 1.....K$, if $w_j \leqslant L_i$ do the following.

$$Y_i \leftarrow Y_i \cup \Big\{ \big(C_1, \ldots, C_{i-1}, \max\{C_i + b_j,\ A_i + a_j + b_j\}, C_{i+1}, \ldots, C_K, \\ A_1, \ldots, A_{i-1}, A_i + a_j, A_{i+1}, \ldots, A_K, \\ B_1, \ldots, B_{i-1}, B_i + b_j, B_{i+1}, \ldots, B_K\big) \Big\}$$

Merge $i = 1, ..., K$, to obtain $\mathscr{H}^{(j)}$. If we get the same tuples during the merge, store only one of them.

Once all jobs are assigned and each block $\mathcal{B}_i$ is summarized by its parameters (A_i, B_i, C_i), the overall makespan $C_{\max}$ is obtained by concatenating the blocks in order of increasing index. The procedure is given below:

In the update of T_B, the term $T_A - A_i$ is the start time of $\mathcal{B}_i$ on M_1, so $T_A - A_i + C_i$ is the completion time of $\mathcal{B}_i$ on M_2 when M_2 is free, while $T_B + B_i$ accounts for the case where M_2 is the bottleneck.

Algorithm 1

Require: Block parameters $(A_i,\ B_i\ )$ for $i = 1, \ldots, K$.

Ensure: Completion times $C_{\max}$.

1: $T_A \leftarrow 0, \quad T_B \leftarrow 0$
2: **for** $i = 1$ to K **do**
3: $\quad T_A^{new} \leftarrow T_A^{old} + A_i$
4: $\quad T_B^{new} \leftarrow \max\{\ T_A^{new} - A_i + C_i,\ T_B^{old} + B_i\ \}$
5: **end for**
6: **return** T_B

Algorithm 2

Require: Jobs $\mathcal{J}$ with (a_j, b_j, w_j), numbered $1, \ldots, n$ in Johnson order;

Ensure: The optimal objective value $WC^*_{\max}$.

$\mathscr{H}^{(0)} \leftarrow \{(0, \ldots, 0)\}$
for $j = 1$ to n **do**
  $Y_i \leftarrow \emptyset$ for $i = 1, \ldots, K$
  for each $(C_1, \ldots, C_K;\ A_1, \ldots, A_K;\ B_1, \ldots, B_K;\ L_1, \ldots, L_K) \in \mathscr{H}^{(j-1)}$ **do**
    for $i = 1$ to K **do**
      if $w_j \le L_i$ **then**
        $C'_i \leftarrow \max\{\, C_i + b_j,\ A_i + a_j + b_j \,\}$
        $A'_i \leftarrow A_i + a_j$
        $B'_i \leftarrow B_i + b_j$
        Add $(\ldots, C'_i, \ldots;\ \ldots, A'_i, \ldots;\ \ldots, B'_i, \ldots;\ )$ to Y_i
      end if
    end for
  end for
  $\mathscr{H}^{(j)} \leftarrow \textsc{Merge}(Y_1, \ldots, Y_K)$ by Lemma 4.1
end for
$C_{\max} \leftarrow +\infty$
for each state $S \in \mathscr{H}$ **do**
  $T_B \leftarrow$ **Algorithm 1**$\big((A_1,\, B_1,\, C_1),\ \ldots,\ (A_K,\, B_K,\, C_K)\big)$
  $C_{\max} \leftarrow \min\{\, C_{\max},\ T_B \,\}$
end for
return $C_{\max}$

The algorithm above computes the optimal makespan $C_{\max}$. To obtain the weighted objective $WC_{\max}$ from a state $S \in \mathscr{H}$, let $T_A^{(i)}$ and $T_B^{(i)}$ be the completion times on M_1 and M_2 after concatenating blocks $\mathcal{B}_1$ through $\mathcal{B}_i$ via the Algorithmic 1. Since blocks are concatenated in non-increasing weight order, $T_B^{(i)}$ is precisely the completion time of the last job in $\mathcal{B}_i$. The weighted completion time of the schedule corresponding to S is therefore

$$WC_{\max}(S) = \max_{i=1,\ldots,K} L_i \cdot T_B^{(i)},$$

where L_i is the weight recorded for $\mathcal{B}_i$ (the weight of its last job). Taking the minimum over all states in $\mathscr{H}$ yields the optimal objective value:

$$WC^*_{\max} = \min_{S \in \mathscr{H}} \max_{i=1,\ldots,K} L_i \cdot T_B^{(i)}.$$

Theorem 4.1. *Algorithm 2 solves $F2 \,\|\, WC_{\max}$ optimally in $O\big(n \cdot K \cdot |\mathscr{H}|_{\max}\big)$ time, where $|\mathscr{H}|_{\max} \le K^K \cdot V^{\,3K-2}$, $V = \max\big\{\sum_{j=1}^n a_j,\ \sum_{j=1}^n b_j\big\}$. For constant K, this is $O(n \cdot V^{\,3K-2})$.*

Proof. Each state in $\mathscr{H}^{(j)}$ is a $4K$-tuple $(C_1, \ldots, C_K;\ A_1, \ldots, A_K;\ B_1, \ldots, B_K;\ L_1, \ldots, L_K)$. We bound the number of distinct values each component can attain.

1. A_i is the total M_1 processing time of jobs assigned to block $\mathcal{B}_i$, so $A_i \in [0,\ V]$. Since the job sequence is fixed and the j jobs under consideration form a prefix, $\sum_{i=1}^{K} A_i$ equals the sum of a_j over that prefix; consequently at most $K-1$ of the A_i are independent, contributing $\leq V^{\,K-1}$ distinct A-vectors.

2. Symmetrically, $\sum_{i=1}^{K} B_i$ is the prefix sum of the b_j's, so the B-vectors contribute $\leq V^{\,K-1}$ distinct combinations.

3. C_i is the makespan of $\mathcal{B}_i$ scheduled in isolation according to Johnson's rule. Since $C_i \leq A_i + B_i \leq 2V$, the C-vector contributes $\leq (2V)^K$ distinct combinations.

4. L_i is the weight of the last job in $\mathcal{B}_i$ and takes values from $\{w^{(1)}, \ldots, w^{(K)}\}$, contributing $\leq K^K$ distinct L-vectors.

Multiplying these bounds yields

$$|\mathscr{H}^{(j)}| \leq K^K \cdot (2V)^K \cdot V^{\,2K-2} = O\big(K^K \cdot 2^K \cdot V^{\,3K-2}\big).$$

In iteration j, the algorithm takes each state in $\mathscr{H}^{(j-1)}$ and each block $i \in \{1, \ldots, K\}$, computes the updated tuple in $O(1)$ time, and inserts it into Y_i. The merge step collects the K sets $Y_1, \ldots, Y_K$ and removes duplicates, which can be done in time proportional to the total number of generated tuples. Hence iteration j costs $O(K \cdot |\mathscr{H}^{(j-1)}|)$, and the n iterations together cost $O(n \cdot K \cdot |\mathscr{H}|_{\max})$.

The final concatenation (Algorithm 1) evaluates each state in $\mathscr{H}^{(n)}$ in $O(K)$ time, which is dominated by the DP phase. When K is fixed, $K^K \cdot 2^K$ is constant, giving $O(n \cdot V^{\,3K-2})$. □

4.2 FPTAS

5 Conclusion

References

Baker, K. R. (1974). *Introduction to Sequencing and Scheduling.* Wiley, New York.

Hall, L. A. (1994). A polynomial approximation scheme for a constrained flow-shop scheduling problem. *Mathematics of Operations Research*, 19(1):68–85.

Johnson, S. M. (1954). Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68.